

CAHIER DES CHARGES

Conception et développement d'une plateforme E-Commerce

Architecture Microservices avec Spring Boot et Angular

Master 1 — Systèmes d'Information

Modules : Développement Backend (Spring Boot) & Développement Frontend (Angular)

Équipe projet

Adama Seck

Abdoulaye Niang

Aissatou Diagne

Arame Bacar Cissé

Racine Sabaly

Année académique 2025 - 2026

Table des matières

1. Présentation générale du projet

1.1 Contexte

Le commerce électronique s'est imposé, en l'espace d'une dizaine d'années, comme l'un des principaux vecteurs de croissance économique à l'échelle mondiale. Les habitudes de consommation ont profondément évolué : les clients recherchent désormais des expériences d'achat rapides, personnalisées et accessibles depuis n'importe quel support, tout en exigeant un niveau élevé de fiabilité et de sécurité dans le traitement de leurs données personnelles et de leurs paiements.

Dans ce contexte, les entreprises qui souhaitent rester compétitives doivent disposer de plateformes capables d'absorber une charge variable, d'évoluer rapidement au gré des besoins métier, et de garantir une disponibilité quasi continue du service. Les architectures monolithiques traditionnelles, bien qu'elles aient longtemps constitué la norme, montrent aujourd'hui leurs limites face à ces exigences : couplage fort entre les modules, difficulté à faire évoluer une partie du système sans impacter l'ensemble, déploiements lourds et risqués, et scalabilité limitée.

C'est dans cette optique que le présent projet a été conçu. Réalisé dans le cadre des enseignements de Master 1 en Systèmes d'Information, il vise à mettre en pratique les concepts d'architecture microservices, de développement backend avec Spring Boot et de développement frontend avec Angular, à travers la conception et la réalisation d'une plateforme e-commerce moderne, modulaire et évolutive.

1.2 Problématique

La problématique centrale à laquelle ce projet cherche à répondre peut être formulée ainsi : comment concevoir une plateforme de vente en ligne capable de gérer de façon indépendante la gestion des utilisateurs, la gestion du catalogue produit, la gestion des commandes et la gestion des paiements, tout en garantissant la cohérence des données, la sécurité des échanges et une expérience utilisateur fluide, dans un contexte où chaque domaine métier doit pouvoir évoluer, être déployé et être maintenu séparément des autres ?

Cette problématique soulève plusieurs sous-questions techniques et organisationnelles : comment répartir les responsabilités entre les différents services sans dupliquer inutilement les données ni créer de dépendances fortes entre eux ? Comment garantir qu'un visiteur non authentifié puisse consulter librement le catalogue tout en étant systématiquement redirigé vers l'authentification dès qu'il souhaite initier un acte d'achat ? Comment assurer la traçabilité et l'intégrité d'une commande, de sa création jusqu'à la génération de la facture, en passant par la réservation du stock et la validation du paiement, alors que ces étapes sont prises en charge par des microservices distincts communiquant via le réseau ?

Ce sont ces questions qui ont guidé l'ensemble des choix d'architecture, de modélisation des données et de conception des flux métier détaillés dans ce document.

1.3 Solution proposée

La solution retenue par le groupe consiste à développer une plateforme e-commerce complète organisée autour d'une architecture microservices, dans laquelle chaque domaine fonctionnel majeur est isolé au sein d'un service autonome disposant de sa propre base de données PostgreSQL, conformément au principe « Database per Service ». Quatre microservices métier composent le cœur du système : un service de gestion des utilisateurs et de l'authentification, un service de gestion du catalogue (produits, catégories, stocks et prix de gros), un service de gestion des commandes (panier, commande, facturation, historique) et un service de gestion des paiements.

L'ensemble de ces services est exposé au frontend Angular à travers une API Gateway unique, qui joue le rôle de point d'entrée centralisé, de routeur des requêtes vers les microservices appropriés, et de première ligne de sécurité via la validation des jetons JWT. Les communications inter-services, lorsqu'elles sont nécessaires,

s'appuient sur des appels REST synchrones réalisés avec OpenFeign, complétés par des mécanismes de résilience assurés par Resilience4J (circuit breaker, retry, fallback) afin de limiter la propagation des pannes.

L'ensemble de l'écosystème applicatif — microservices, bases de données et API Gateway — est conteneurisé avec Docker et orchestré localement à l'aide de Docker Compose, ce qui permet de reproduire un environnement proche d'un déploiement réel tout en restant simple à installer et à démontrer dans un cadre académique.

1.4 Objectifs généraux

Le projet poursuit un objectif général double. D'une part, il s'agit de concevoir et développer une application e-commerce fonctionnelle, complète et cohérente, couvrant l'ensemble du parcours d'achat, depuis la consultation du catalogue jusqu'à la réception de la facture. D'autre part, il s'agit de mettre en œuvre, de manière rigoureuse et documentée, les compétences techniques acquises durant les modules de développement Backend avec Spring Boot et de développement Frontend avec Angular, en particulier les notions d'inversion de contrôle, d'injection de dépendances, de persistance avec Spring Data JPA, de sécurisation par JWT, et de construction d'API REST au sein d'une architecture distribuée.

1.5 Objectifs spécifiques

Au-delà de l'objectif général, plusieurs objectifs spécifiques et mesurables ont été fixés par le groupe :

- Mettre en place une architecture microservices comportant au minimum quatre services métier indépendants, chacun disposant de sa propre base de données PostgreSQL.
- Développer une API Gateway assurant le routage des requêtes et la vérification préalable des jetons JWT avant transmission aux microservices.
- Implémenter un mécanisme d'authentification et d'autorisation basé sur JWT avec gestion des rôles (visiteur, utilisateur, administrateur).
- Concevoir un catalogue produit permettant la consultation libre par tout visiteur, avec recherche, filtrage et gestion des prix de gros par paliers de quantité.
- Mettre en œuvre un processus complet de commande incluant la gestion du panier, la réservation temporaire du stock, le paiement et la génération automatique de la facture.
- Garantir la résilience des communications inter-services grâce à Resilience4J (circuit breaker, retry, time limiter).
- Documenter l'ensemble des API REST à l'aide de Swagger/OpenAPI et valider leur bon fonctionnement à l'aide de collections Postman.
- Conteneuriser l'ensemble des composants applicatifs avec Docker et orchestrer leur démarrage avec Docker Compose.
- Assurer un partage de code structuré et traçable via un dépôt GitHub organisé par microservice.

1.6 Bénéfices attendus

Sur le plan pédagogique, ce projet permet au groupe de confronter les concepts théoriques enseignés en cours à la réalité d'un développement d'envergure, incluant les difficultés propres aux systèmes distribués : gestion de la cohérence des données entre services, communication réseau, sécurité transversale, et coordination d'équipe autour de composants interdépendants. Il permet également de renforcer la maîtrise des outils professionnels du marché tels que Spring Boot, Angular, Docker et Git, largement utilisés en entreprise.

Sur le plan fonctionnel, la plateforme développée apporte une valeur démonstrative forte : elle illustre de façon concrète comment une architecture moderne peut répondre aux exigences réelles d'un site marchand, à savoir la disponibilité, la sécurité des transactions, la gestion fine des stocks et des tarifs, ainsi que la

capacité à faire évoluer chaque brique indépendamment des autres, ce qui constitue un atout majeur par rapport à une architecture monolithique classique.

1.7 Public cible

La plateforme s'adresse, dans son scénario d'usage, à trois catégories de personnes. La première est constituée des visiteurs occasionnels, c'est-à-dire toute personne consultant le site sans y être identifiée, dans une logique de découverte du catalogue avant décision d'achat. La deuxième catégorie regroupe les clients enregistrés, qui disposent d'un compte personnel leur permettant de finaliser des achats, de suivre leurs commandes et de consulter leurs factures. La troisième catégorie correspond au personnel administratif de la plateforme, chargé de la gestion du catalogue, des tarifs, des commandes et des comptes utilisateurs.

1.8 Utilisateurs du système

Trois profils utilisateurs distincts sont pris en charge par le système, chacun disposant d'un ensemble de droits et de fonctionnalités spécifiques détaillés dans la section relative à l'analyse des besoins : le Visiteur, l'Utilisateur authentifié et l'Administrateur. Cette distinction des rôles est portée techniquement par le mécanisme d'autorisation basé sur les jetons JWT, chaque jeton embarquant les informations de rôle nécessaires à la vérification des droits d'accès au niveau de l'API Gateway et de chaque microservice.

2. Analyse des besoins

Cette section a pour objectif de recenser de manière exhaustive les besoins fonctionnels du système, en les organisant par profil d'utilisateur, ainsi que d'établir la liste des règles de comportement attendues du système face à chaque action. Cette analyse constitue le socle à partir duquel les entités du modèle de données, les endpoints des API REST et les règles métier ont été définis dans les sections suivantes.

2.1 Besoins fonctionnels

2.1.1 Le Visiteur

Le visiteur désigne toute personne accédant à la plateforme sans authentification préalable. Le parti pris de conception retenu par le groupe est de laisser un accès entièrement libre à la consultation du catalogue, dans une logique commerciale classique où la découverte des produits ne doit jamais être bridée par une contrainte de connexion. En revanche, toute action impliquant un engagement (ajout au panier, commande, paiement) déclenche une redirection vers l'authentification.

Le visiteur peut :

- Consulter la liste des catégories de produits disponibles.
- Consulter la liste des produits, avec pagination.
- Consulter la fiche détaillée d'un produit (description, prix, images, stock disponible, prix de gros éventuels).
- Rechercher un produit par mot-clé (nom, description).
- Filtrer les produits par catégorie, par tranche de prix ou par disponibilité en stock.

Le visiteur ne peut pas :

- Ajouter un produit au panier.
- Passer une commande.
- Effectuer un paiement.
- Consulter un quelconque historique ou une facture.

Règle de redirection : lorsque le visiteur clique sur le bouton « Ajouter au panier » ou sur le bouton « Commander », le frontend Angular intercepte l'action, constate l'absence de jeton JWT valide dans le contexte applicatif, et redirige automatiquement l'utilisateur vers la page de connexion ou de création de compte. Une fois l'authentification réussie, l'action initialement demandée (ajout du produit concerné) est rejouée automatiquement afin de ne pas pénaliser l'expérience utilisateur.

2.1.2 L'Utilisateur authentifié

L'utilisateur authentifié est un visiteur ayant créé un compte et s'étant connecté avec succès. Il hérite de l'ensemble des droits du visiteur et bénéficie en complément des fonctionnalités transactionnelles de la plateforme.

- Créer un compte (inscription) avec validation des champs obligatoires.
- Se connecter à l'aide de son email et de son mot de passe.
- Modifier les informations de son profil (nom, prénom, téléphone, adresse, avatar).
- Modifier son mot de passe, avec vérification de l'ancien mot de passe.
- Consulter, rechercher et filtrer les produits (hérité du visiteur).
- Ajouter un ou plusieurs produits à son panier.
- Modifier la quantité d'un article présent dans le panier.
- Supprimer un article du panier.

- Passer une commande à partir du contenu de son panier.
- Choisir ou saisir une adresse de livraison.
- Choisir un mode de paiement parmi ceux proposés par la plateforme.
- Effectuer le paiement de sa commande.
- Consulter la liste et le détail de ses commandes passées.
- Consulter et télécharger la facture associée à une commande payée.
- Consulter l'historique complet de ses achats.

2.1.3 L'Administrateur

L'administrateur dispose d'un accès étendu, orienté vers la gestion opérationnelle de la plateforme. Ses droits sont organisés en cinq grands domaines fonctionnels détaillés ci-dessous.

a) Gestion des produits

- Ajouter un nouveau produit (informations, images, stock initial, catégorie).
- Modifier les informations d'un produit existant.
- Supprimer un produit, sous réserve qu'il ne soit associé à aucune commande déjà payée.
- Masquer un produit (le rendre invisible du catalogue public sans le supprimer).
- Afficher (réactiver la visibilité) un produit précédemment masqué.
- Réapprovisionner le stock d'un produit.

b) Gestion des prix de gros

Le système de prix de gros permet à l'administrateur de définir, pour un produit donné, plusieurs paliers de quantité associés chacun à un tarif préférentiel. Par exemple, pour un produit dont le prix unitaire normal est de 5 000 FCFA, l'administrateur peut définir qu'à partir de 10 unités commandées, le prix unitaire descend à 4 500 FCFA, et qu'à partir de 20 unités, il descend à 4 000 FCFA. Lors du calcul du montant d'une ligne de commande, le système détermine automatiquement, parmi les paliers actifs du produit, celui dont le seuil de quantité minimale est le plus élevé tout en restant inférieur ou égal à la quantité commandée, et applique le tarif correspondant.

- Ajouter un palier de prix de gros à un produit (quantité minimale, prix associé).
- Modifier un palier existant.
- Supprimer un palier.
- Activer ou désactiver un palier sans le supprimer.

c) Gestion des catégories

- Ajouter une nouvelle catégorie de produits.
- Modifier le nom ou la description d'une catégorie.
- Supprimer une catégorie, sous réserve qu'aucun produit actif n'y soit rattaché.
- Masquer une catégorie : tous les produits qui lui sont rattachés deviennent alors invisibles du catalogue public.
- Afficher une catégorie précédemment masquée.

d) Gestion des commandes

- Consulter la liste de toutes les commandes passées sur la plateforme, avec filtres par statut, date ou client.
- Consulter le détail complet d'une commande (lignes de commande, montants, adresse, statut, paiement).

- Modifier le statut d'une commande lorsque la transition est autorisée par la machine à états définie dans les règles métier (par exemple faire passer une commande de « En préparation » à « Expédiée »).

e) Gestion des utilisateurs

- Consulter la liste des utilisateurs inscrits, avec recherche et filtres.
- Consulter le détail d'un utilisateur (informations, historique de commandes).
- Activer un compte utilisateur.
- Désactiver un compte utilisateur, ce qui lui interdit immédiatement toute nouvelle connexion.

f) Tableau de bord

Le tableau de bord administrateur agrège des indicateurs clés de pilotage de l'activité, calculés à partir des données exposées par les différents microservices :

- Nombre total de produits actifs et inactifs.
- Nombre total de commandes, réparties par statut (en attente, payées, expédiées, annulées).
- Chiffre d'affaires total et chiffre d'affaires sur une période donnée.
- Nombre total d'utilisateurs inscrits, actifs et désactivés.
- Nombre et montant total des paiements réalisés avec succès.

2.2 Matrice fonctionnalités / rôles

Fonctionnalité	Visiteur	Utilisateur	Administrateur
Consulter le catalogue	Oui	Oui	Oui
Rechercher / filtrer	Oui	Oui	Oui
Ajouter au panier	Non (redirection)	Oui	Non applicable
Passer commande	Non	Oui	Non applicable
Payer	Non	Oui	Non applicable
Consulter factures	Non	Oui (les siennes)	Oui (toutes)
Gérer produits/catégories	Non	Non	Oui
Gérer prix de gros	Non	Non	Oui
Gérer utilisateurs	Non	Non	Oui
Consulter tableau de bord	Non	Non	Oui

3. Besoins non fonctionnels

Au-delà des fonctionnalités visibles par les utilisateurs, la qualité d'une plateforme e-commerce repose largement sur des exigences transverses qui conditionnent sa robustesse, sa sécurité et sa capacité à évoluer dans le temps. Ces exigences non fonctionnelles ont été prises en compte dès la phase de conception de l'architecture.

3.1 Performance

Le système doit être en mesure de répondre aux requêtes de consultation du catalogue en un temps raisonnable, y compris lorsque le nombre de produits référencés devient important. Les listes de produits sont paginées côté backend afin d'éviter de transférer des volumes de données inutiles au frontend. Les endpoints les plus sollicités, comme la consultation du catalogue, sont conçus pour minimiser le nombre de jointures et de traitements coûteux. Les échanges inter-services via OpenFeign sont limités au strict nécessaire afin de ne pas dégrader les temps de réponse perçus par l'utilisateur final.

3.2 Disponibilité

Chaque microservice étant indépendant et disposant de sa propre base de données, l'indisponibilité d'un service n'entraîne pas nécessairement l'arrêt complet de la plateforme. Par exemple, une panne du service de paiement ne doit pas empêcher un visiteur de continuer à consulter le catalogue. Cette isolation des pannes est renforcée par les mécanismes de circuit breaker fournis par Resilience4J, qui permettent de dégrader gracieusement le service plutôt que de propager une erreur en cascade.

3.3 Fiabilité

La fiabilité du système repose sur la cohérence des données au sein de chaque microservice, garantie par les transactions gérées par Spring Data JPA et Hibernate, ainsi que sur la validation systématique des données entrantes à l'aide de Bean Validation. Les opérations critiques, telles que la réservation de stock ou la validation d'un paiement, sont conçues pour être idempotentes autant que possible, afin d'éviter les incohérences en cas de nouvelle tentative suite à une erreur réseau.

3.4 Sécurité

La sécurité constitue une exigence transversale majeure du projet, détaillée intégralement dans la section 9 dédiée. Elle s'appuie notamment sur l'authentification par jetons JWT, le chiffrement des mots de passe avec BCrypt, la validation stricte des entrées utilisateur et la gestion fine des rôles et permissions.

3.5 Maintenabilité

Le découpage en microservices favorise intrinsèquement la maintenabilité, chaque équipe pouvant faire évoluer son service sans impacter directement le code des autres. Chaque microservice suit une architecture en couches (contrôleur, service, repository, entité) clairement séparées, avec une convention de nommage homogène décrite dans la section relative aux contraintes techniques, ce qui facilite la compréhension et la reprise du code par un tiers.

3.6 Scalabilité

L'architecture retenue permet, en théorie de déploiement, de faire évoluer indépendamment la capacité de chaque microservice en fonction de sa charge réelle. Le service de gestion du catalogue, généralement le plus sollicité en lecture, pourrait par exemple être répliqué en plusieurs instances derrière un équilibreur de charge sans nécessiter la moindre modification des autres services, dès lors que l'API Gateway route les requêtes de façon transparente.

3.7 Extensibilité

La séparation stricte des responsabilités entre microservices permet d'envisager l'ajout de nouveaux services (par exemple un service de notifications, un service d'avis clients ou un service de recommandation) sans remettre en cause l'existant, à condition de respecter les contrats d'API déjà établis et le principe d'indépendance des bases de données.

3.8 Portabilité

La conteneurisation de l'ensemble des composants avec Docker garantit la portabilité de la solution : l'application peut être déployée de façon identique sur la machine d'un développeur, sur un serveur de démonstration ou, à terme, sur une infrastructure cloud, sans dépendre de l'environnement d'exécution sous-jacent.

3.9 Ergonomie

L'interface Angular est conçue pour offrir un parcours d'achat simple et intuitif, avec un minimum d'étapes entre la découverte d'un produit et la validation d'une commande. Les messages d'erreur et de confirmation sont explicites, et la redirection vers l'authentification, lorsqu'elle est nécessaire, est présentée de manière non intrusive afin de ne pas frustrer le visiteur.

3.10 Compatibilité

Le frontend est développé pour être compatible avec les navigateurs modernes (Chrome, Firefox, Edge) dans leurs versions récentes, et respecte les standards d'accessibilité de base. Les API REST exposées respectent les standards HTTP et sont documentées via Swagger/OpenAPI, ce qui garantit leur interopérabilité avec tout client tiers respectant le contrat d'interface.

3.11 Journalisation

Chaque microservice journalise ses opérations significatives (authentification, création de commande, validation de paiement, erreurs applicatives) afin de permettre le diagnostic a posteriori d'un incident. Les journaux incluent systématiquement un horodatage, le niveau de sévérité et un identifiant de corrélation lorsque cela est pertinent pour suivre une requête à travers plusieurs microservices.

3.12 Surveillance

Des points de contrôle de santé (endpoints d'actuator) sont exposés par chaque microservice, permettant de vérifier à tout moment son état de fonctionnement ainsi que la disponibilité de sa base de données. Ces points de contrôle constituent la base d'une supervision applicative, qui pourrait être complétée à terme par un tableau de bord centralisé.

3.13 Tolérance aux pannes

Les appels inter-services réalisés via OpenFeign sont enveloppés par des mécanismes Resilience4J : un circuit breaker interrompt temporairement les appels vers un service détecté comme défaillant, un mécanisme de retry retente automatiquement un appel ayant échoué pour une raison transitoire, et un système de fallback permet de renvoyer une réponse dégradée mais cohérente plutôt qu'une erreur brute à l'utilisateur final.

3.14 Sauvegardes

Chaque base de données PostgreSQL, propre à un microservice, fait l'objet d'une politique de sauvegarde régulière, indépendante des autres bases. Cette indépendance présente l'avantage de permettre une restauration ciblée d'un seul domaine métier en cas d'incident, sans devoir restaurer l'intégralité du système.

4. Architecture générale

L'architecture de la plateforme repose sur une organisation en couches successives, allant du client Angular jusqu'aux bases de données PostgreSQL, en passant par une passerelle API et un ensemble de microservices Spring Boot indépendants. Le schéma logique général de circulation d'une requête est le suivant :

Frontend Angular → API Gateway → Microservice concerné → Base PostgreSQL dédiée

4.1 Frontend Angular

Le frontend, développé avec Angular 20, constitue l'unique point de contact avec l'utilisateur final, qu'il s'agisse d'un visiteur, d'un utilisateur authentifié ou d'un administrateur. Il est organisé en modules fonctionnels (catalogue, authentification, panier, commandes, administration) et communique exclusivement avec l'API Gateway, jamais directement avec un microservice. Le jeton JWT obtenu lors de la connexion est conservé côté client et automatiquement joint à chaque requête nécessitant une authentification, via un intercepteur HTTP Angular.

4.2 API Gateway

L'API Gateway constitue le point d'entrée unique de l'ensemble du système. Son rôle est triple : router chaque requête entrante vers le microservice compétent en fonction de l'URL demandée, vérifier la présence et la validité du jeton JWT pour les routes protégées avant de transmettre la requête, et exposer une interface unique et stable au frontend, masquant ainsi la complexité et la localisation réelle des microservices sous-jacents. L'API Gateway ne contient aucune logique métier ; elle se limite à des responsabilités transversales de routage et de sécurité de premier niveau.

4.3 Microservices métier

Le système est composé de quatre microservices métier, chacun étant responsable d'un domaine fonctionnel clairement délimité et disposant de sa propre base de données PostgreSQL, conformément au principe Database per Service. Aucune base de données n'est partagée entre deux microservices : lorsqu'un service a besoin d'une information détenue par un autre, il l'obtient via un appel REST synchrone réalisé avec OpenFeign, jamais par accès direct à une base tierce.

- User/Auth Service : gestion des comptes, de l'authentification et des rôles.
- Catalog Service : gestion des produits, des catégories, des stocks et des prix de gros.
- Order Service : gestion du panier, des commandes, des lignes de commande et des factures.
- Payment Service : gestion des paiements et de leur validation.

4.4 Bases de données PostgreSQL dédiées

Chaque microservice dispose de sa propre instance de base de données PostgreSQL, isolée des autres tant au niveau du schéma que de l'instance de conteneur Docker qui l'héberge. Ce choix garantit l'autonomie totale de chaque service : son schéma de données peut évoluer sans concertation avec les autres équipes, et une panne ou une migration sur une base n'affecte jamais les autres bases du système.

4.5 Communication inter-services : OpenFeign

Lorsqu'un microservice a besoin d'une donnée détenue par un autre service — par exemple, lorsque l'Order Service doit vérifier la disponibilité du stock d'un produit auprès du Catalog Service, ou lorsqu'il doit vérifier la validité d'un utilisateur auprès du User Service — cet appel est réalisé via un client OpenFeign déclaratif. OpenFeign permet de définir une interface Java représentant le contrat d'API du service distant, l'implémentation technique de l'appel HTTP étant entièrement prise en charge par le framework.

4.6 Résilience : Resilience4J

Chaque appel OpenFeign entre microservices est protégé par les mécanismes de la bibliothèque Resilience4J. Un circuit breaker surveille le taux d'échec des appels vers un service donné et interrompt temporairement les appels lorsque ce taux dépasse un seuil défini, évitant ainsi de solliciter inutilement un service déjà en difficulté. Un mécanisme de retry retente automatiquement, avec un délai croissant, un appel ayant échoué pour une cause transitoire (timeout réseau, indisponibilité momentanée). Enfin, une méthode de fallback est définie pour chaque appel critique, permettant de renvoyer une réponse par défaut cohérente plutôt qu'une erreur brute lorsque le service distant reste indisponible.

4.7 Sécurité transversale : JWT

L'authentification de la plateforme repose sur des jetons JWT (JSON Web Token) émis par le User/Auth Service lors de la connexion. Ce jeton, signé et à durée de vie limitée, est ensuite présenté par le client à chaque requête vers l'API Gateway, qui en vérifie la validité et la signature avant de transmettre la requête au microservice cible avec les informations d'identité et de rôle extraites du jeton.

4.8 Conteneurisation : Docker et Docker Compose

Chaque microservice, chaque base de données et l'API Gateway sont packagés sous forme d'images Docker indépendantes. Un fichier Docker Compose unique orchestre le démarrage de l'ensemble de ces conteneurs, définit le réseau interne permettant aux services de communiquer entre eux par leur nom logique, et déclare les volumes persistants nécessaires à la conservation des données des bases PostgreSQL entre deux redémarrages.

4.9 Schéma logique récapitulatif

Le schéma logique de l'architecture peut être résumé de la manière suivante : le navigateur exécute l'application Angular, laquelle adresse toutes ses requêtes HTTP à l'API Gateway sur un port unique exposé publiquement. L'API Gateway route ensuite chaque requête vers le microservice compétent (User/Auth, Catalog, Order ou Payment), chacun exposant son propre port interne au sein du réseau Docker. Chaque microservice dialogue exclusivement avec sa base PostgreSQL dédiée, et, lorsque nécessaire, avec les autres microservices via des appels OpenFeign encapsulés par Resilience4J.

5. Description complète des microservices

5.1 User/Auth Service

Responsabilités

Le User/Auth Service centralise l'ensemble des opérations relatives à l'identité des utilisateurs : inscription, authentification, gestion du profil, gestion du mot de passe et émission des jetons JWT. Il est également l'autorité de référence quant au rôle et au statut (actif ou désactivé) de chaque compte.

Base de données

Ce service dispose de sa propre base PostgreSQL, nommée par convention userdb, contenant exclusivement les tables relatives aux comptes utilisateurs et à leurs informations associées (adresses, avatar).

Entités principales

- Utilisateur (User)
- Adresse
- Rôle

API exposées (aperçu)

Inscription, connexion, rafraîchissement de jeton, consultation et modification du profil, changement de mot de passe, ainsi que les opérations d'administration des comptes (liste, activation, désactivation). Le détail complet des endpoints figure en section 7.

Flux principal

Lors de l'inscription, le service valide les champs obligatoires, vérifie l'unicité de l'adresse email, chiffre le mot de passe avec BCrypt puis enregistre le compte avec le rôle « Utilisateur » par défaut. Lors de la connexion, le service vérifie les identifiants, contrôle que le compte est actif, puis génère un jeton JWT signé contenant l'identifiant, l'email et le rôle de l'utilisateur.

Sécurité

Le service applique une politique de mot de passe minimale (longueur, complexité), un verrouillage temporaire du compte après plusieurs échecs de connexion consécutifs, et ne stocke jamais de mot de passe en clair.

5.2 Catalog Service

Le Catalog Service gère l'intégralité du référentiel produit de la plateforme : les produits eux-mêmes, leur rattachement à une catégorie, l'état de leur stock, et les paliers de prix de gros associés.

- Produits : création, modification, suppression, masquage, affichage, gestion du stock.
- Catégories : création, modification, suppression, masquage, affichage, avec propagation du masquage aux produits rattachés.
- Stocks : suivi de la quantité disponible, décrémentation lors de la confirmation d'un paiement, réservation temporaire lors de la validation d'un panier.
- Prix de gros : définition de paliers de quantité minimale associés à un tarif préférentiel, activation ou désactivation de ces paliers.

Ce service dispose de sa propre base PostgreSQL (catalogdb) contenant les tables Produit, Catégorie et PrixGros. Il expose notamment un endpoint interne, consommé par l'Order Service via OpenFeign, permettant de vérifier la disponibilité d'un produit et de calculer le prix applicable en fonction de la quantité demandée.

5.3 Order Service

L'Order Service prend en charge l'ensemble du cycle de vie d'une commande, depuis la constitution du panier jusqu'à la génération de la facture, en passant par la validation de la commande et le suivi de son statut.

- Panier : ajout, modification de quantité, suppression d'articles, calcul du total.
- Commande : création à partir du panier, choix de l'adresse de livraison et du mode de paiement, gestion du statut.
- Facture : génération automatique après confirmation du paiement, mise à disposition au téléchargement.
- Historique : consultation par l'utilisateur de l'ensemble de ses commandes passées.

Ce service dispose de sa propre base PostgreSQL (orderdb) contenant les tables Panier, PanierItem, Commande, LigneCommande et Facture. Lors de la validation d'une commande, l'Order Service interroge le Catalog Service via OpenFeign pour vérifier la disponibilité du stock et obtenir le prix applicable (normal ou de gros), puis interroge le Payment Service pour initier le processus de paiement.

5.4 Payment Service

Le Payment Service est responsable du traitement des paiements associés à une commande : initiation de la transaction, validation ou rejet, et notification du résultat à l'Order Service afin que ce dernier mette à jour le statut de la commande et déclenche, le cas échéant, la génération de la facture.

- Paiement : enregistrement d'une tentative de paiement liée à une commande.
- Validation : confirmation ou rejet du paiement selon le résultat du traitement.
- Confirmation : notification du résultat à l'Order Service via un appel OpenFeign.
- Mise à jour des commandes : déclenchement, côté Order Service, du passage de la commande au statut « Payée » ou « Paiement échoué ».

Ce service dispose de sa propre base PostgreSQL (paymentdb) contenant la table Paiement, avec une référence logique (identifiant) vers la commande concernée, sans jointure directe en base entre les deux services.

6. Base de données

Cette section décrit le schéma relationnel de chacune des quatre bases PostgreSQL, chacune étant strictement rattachée à un unique microservice, conformément au principe Database per Service. Les clés étrangères inter-microservices n'existent jamais physiquement en base : lorsqu'une table doit référencer une entité d'un autre service (par exemple une commande référençant un utilisateur), seule la clé (l'identifiant) est stockée, sans contrainte de clé étrangère, la cohérence étant vérifiée applicativement via un appel OpenFeign.

6.1 Base userdb — User/Auth Service

Table USERS

Colonne	Type	Description
id	BIGSERIAL / PK	Identifiant unique de l'utilisateur
nom	VARCHAR(100)	Nom de famille
prenom	VARCHAR(100)	Prénom
email	VARCHAR(150), UNIQUE	Adresse email, identifiant de connexion
password	VARCHAR(255)	Mot de passe chiffré avec BCrypt
telephone	VARCHAR(20)	Numéro de téléphone
adresse	VARCHAR(255)	Adresse postale principale
avatar_url	VARCHAR(255)	URL de la photo de profil
profil	VARCHAR(20)	Rôle : VISITEUR, UTILISATEUR ou ADMIN
is_active	BOOLEAN	Indique si le compte est actif
date_creation	TIMESTAMP	Date de création du compte
date_modification	TIMESTAMP	Date de dernière modification

6.2 Base catalogdb — Catalog Service

Table CATEGORIES

Colonne	Type	Description
id	BIGSERIAL / PK	Identifiant unique de la catégorie
nom	VARCHAR(100)	Nom de la catégorie
description	TEXT	Description de la catégorie
statut	VARCHAR(20)	ACTIF ou MASQUE

Table PRODUITS

Colonne	Type	Description
id	BIGSERIAL / PK	Identifiant unique du produit
nom	VARCHAR(150)	Nom du produit

Colonne	Type	Description
description	TEXT	Description détaillée
prix	NUMERIC(12,2)	Prix unitaire normal
stock	INTEGER	Quantité disponible en stock
statut	VARCHAR(20)	ACTIF ou MASQUE
image_principale	VARCHAR(255)	URL de l'image principale
images_secondaires	TEXT	Liste d'URLs (JSON ou séparées par virgule)
categorie_id	BIGINT / FK	Référence vers CATEGORIES.id
date_creation	TIMESTAMP	Date de création
date_modification	TIMESTAMP	Date de dernière modification

Table PRIX_GROS

Colonne	Type	Description
id	BIGSERIAL / PK	Identifiant unique du palier
produit_id	BIGINT / FK	Référence vers PRODUITS.id
quantite_minimale	INTEGER	Seuil de quantité déclenchant le tarif
prix	NUMERIC(12,2)	Prix unitaire applicable au-delà du seuil
statut	VARCHAR(20)	ACTIF ou INACTIF

6.3 Base orderdb — Order Service

Table PANIER

Colonne	Type	Description
id	BIGSERIAL / PK	Identifiant unique du panier
utilisateur_id	BIGINT	Référence logique vers l'utilisateur (User Service)
date_creation	TIMESTAMP	Date de création du panier

Table PANIER_ITEM

Colonne	Type	Description
id	BIGSERIAL / PK	Identifiant unique de la ligne de panier
panier_id	BIGINT / FK	Référence vers PANIER.id
produit_id	BIGINT	Référence logique vers PRODUITS (Catalog Service)
quantite	INTEGER	Quantité sélectionnée
prix_unitaire	NUMERIC(12,2)	Prix unitaire retenu au moment de l'ajout

Table COMMANDE

Colonne	Type	Description
id	BIGSERIAL / PK	Identifiant unique de la commande
numero_commande	VARCHAR(30), UNIQUE	Référence lisible de la commande
utilisateur_id	BIGINT	Référence logique vers l'utilisateur
date_commande	TIMESTAMP	Date de création de la commande
etat	VARCHAR(30)	EN_ATTENTE, PAYEE, EXPEDIEE, LIVREE, ANNULEE
adresse_livraison	VARCHAR(255)	Adresse de livraison choisie
mode_paiement	VARCHAR(30)	Mode de paiement choisi
montant_total	NUMERIC(12,2)	Montant total de la commande

Table LIGNE_COMMANDE

Colonne	Type	Description
id	BIGSERIAL / PK	Identifiant unique de la ligne
commande_id	BIGINT / FK	Référence vers COMMANDE.id
produit_id	BIGINT	Référence logique vers PRODUITS
quantite	INTEGER	Quantité commandée
prix_unitaire	NUMERIC(12,2)	Prix unitaire figé au moment de la commande

Table FACTURE

Colonne	Type	Description
id	BIGSERIAL / PK	Identifiant unique de la facture
commande_id	BIGINT / FK	Référence vers COMMANDE.id
numero_facture	VARCHAR(30), UNIQUE	Référence lisible de la facture
date_emission	TIMESTAMP	Date de génération de la facture
montant_total	NUMERIC(12,2)	Montant total facturé

6.4 Base paymentdb — Payment Service

Table PAIEMENT

Colonne	Type	Description
id	BIGSERIAL / PK	Identifiant unique du paiement
commande_id	BIGINT	Référence logique vers COMMANDE (Order Service)
montant	NUMERIC(12,2)	Montant payé
mode_paiement	VARCHAR(30)	Mode de paiement utilisé

Colonne	Type	Description
statut	VARCHAR(20)	EN_COURS, VALIDE, ECHOUE
date_paiement	TIMESTAMP	Date de traitement du paiement

7. Description des API REST

L'ensemble des endpoints décrits ci-dessous est documenté automatiquement via Swagger/OpenAPI sur chaque microservice, et regroupé dans des collections Postman organisées par service, facilitant les tests manuels et l'intégration continue. Les codes de réponse indiqués suivent les conventions HTTP standard.

7.1 User/Auth Service

Méthode	URL	Description	Codes
POST	/api/auth/register	Créer un compte utilisateur	201, 400, 409
POST	/api/auth/login	Authentifier un utilisateur et émettre un JWT	200, 401
POST	/api/auth/refresh-token	Rafraîchir un jeton d'accès expiré	200, 401
GET	/api/users/me	Consulter son propre profil	200, 401
PUT	/api/users/me	Modifier son profil	200, 400, 401
PUT	/api/users/me/password	Modifier son mot de passe	200, 400, 401
GET	/api/users	Lister les utilisateurs (admin)	200, 401, 403
GET	/api/users/{id}	Consulter un utilisateur (admin)	200, 404
PUT	/api/users/{id}/activate	Activer un compte (admin)	200, 404
PUT	/api/users/{id}/deactivate	Désactiver un compte (admin)	200, 404

7.2 Catalog Service

Produits

Méthode	URL	Description	Codes
GET	/api/produits	Lister les produits (public, paginé)	200
GET	/api/produits/{id}	Détail d'un produit (public)	200, 404
GET	/api/produits/recherche?q=	Rechercher un produit (public)	200
GET	/api/produits/filtrer	Filtrer par catégorie/prix/stock (public)	200
POST	/api/produits	Créer un produit (admin)	201, 400, 403
PUT	/api/produits/{id}	Modifier un produit (admin)	200, 404
DELETE	/api/produits/{id}	Supprimer un produit (admin)	204, 409
PUT	/api/produits/{id}/masquer	Masquer un produit (admin)	200, 404
PUT	/api/produits/{id}/afficher	Afficher un produit (admin)	200, 404
PUT	/api/produits/{id}/stock	Réapprovisionner le stock (admin)	200, 404
GET	/api/produits/{id}/disponibilite	Vérifier stock (interne, OpenFeign)	200, 404

Catégories

Méthode	URL	Description	Codes
GET	/api/categories	Lister les catégories (public)	200
POST	/api/categories	Créer une catégorie (admin)	201, 400
PUT	/api/categories/{id}	Modifier une catégorie (admin)	200, 404
DELETE	/api/categories/{id}	Supprimer une catégorie (admin)	204, 409
PUT	/api/categories/{id}/masquer	Masquer une catégorie (admin)	200, 404
PUT	/api/categories/{id}/afficher	Afficher une catégorie (admin)	200, 404

Prix de gros

Méthode	URL	Description	Codes
GET	/api/produits/{id}/prix-gros	Lister les paliers d'un produit	200
POST	/api/produits/{id}/prix-gros	Ajouter un palier (admin)	201, 400
PUT	/api/prix-gros/{id}	Modifier un palier (admin)	200, 404
DELETE	/api/prix-gros/{id}	Supprimer un palier (admin)	204, 404
PUT	/api/prix-gros/{id}/activer	Activer un palier (admin)	200, 404
PUT	/api/prix-gros/{id}/desactiver	Désactiver un palier (admin)	200, 404

7.3 Order Service

Panier

Méthode	URL	Description	Codes
GET	/api/panier	Consulter son panier	200, 401
POST	/api/panier/items	Ajouter un produit au panier	201, 400, 401, 409
PUT	/api/panier/items/{id}	Modifier la quantité d'un article	200, 400, 404
DELETE	/api/panier/items/{id}	Supprimer un article du panier	204, 404

Commande, facture, historique

Méthode	URL	Description	Codes
POST	/api/commandes	Créer une commande à partir du panier	201, 400, 409
GET	/api/commandes	Lister ses commandes (utilisateur)	200, 401

Méthode	URL	Description	Codes
GET	/api/commandes/{id}	Détail d'une commande	200, 403, 404
GET	/api/commandes/admin	Lister toutes les commandes (admin)	200, 403
PUT	/api/commandes/{id}/statut	Modifier le statut (admin)	200, 400, 404
GET	/api/factures/{id}	Consulter une facture	200, 403, 404
GET	/api/factures/{id}/telecharger	Télécharger une facture (PDF)	200, 404
GET	/api/historique	Consulter son historique d'achats	200, 401

7.4 Payment Service

Méthode	URL	Description	Codes
POST	/api/paiements	Initier un paiement pour une commande	201, 400, 409
GET	/api/paiements/{id}	Consulter le statut d'un paiement	200, 404
PUT	/api/paiements/{id}/valider	Valider un paiement (interne)	200, 404, 409
PUT	/api/paiements/{id}/rejeter	Rejeter un paiement (interne)	200, 404
GET	/api/paiements/commande/{commandeId}	Consulter les paiements d'une commande	200, 404

8. Règles métier

Cette section recense les règles métier applicables à travers l'ensemble du système. Ces règles sont implémentées au niveau de la couche service de chaque microservice et complètent les contraintes de validation portées par les entités.

8.1 Règles relatives aux comptes utilisateurs

- Un utilisateur désactivé ne peut plus se connecter, même avec des identifiants corrects ; le système retourne une erreur explicite indiquant que le compte est désactivé.
- Deux comptes ne peuvent pas partager la même adresse email : toute tentative d'inscription avec un email déjà utilisé est rejetée.
- Le mot de passe est systématiquement chiffré avec BCrypt avant tout enregistrement en base ; il n'est jamais stocké ni retourné en clair.
- Après un nombre défini de tentatives de connexion échouées consécutives, le compte concerné est temporairement verrouillé.

8.2 Règles relatives au catalogue

- Un produit masqué n'est plus visible dans le catalogue public consulté par les visiteurs et utilisateurs, mais reste consultable par l'administrateur.
- Une catégorie masquée entraîne automatiquement le masquage de tous les produits qui lui sont rattachés dans l'affichage public, sans modifier leur statut propre en base.
- La suppression d'une catégorie est refusée tant qu'elle contient des produits actifs.
- La suppression d'un produit est refusée si celui-ci est présent dans au moins une commande déjà payée, afin de préserver l'intégrité de l'historique et des factures.

8.3 Règles relatives aux prix de gros

- Pour une quantité commandée donnée, le système retient automatiquement le palier de prix de gros actif dont le seuil de quantité minimale est le plus élevé parmi ceux inférieurs ou égaux à la quantité commandée.
- En l'absence de palier applicable, ou si tous les paliers d'un produit sont désactivés, le prix normal du produit est appliqué.
- La désactivation d'un palier ne supprime pas son historique d'application sur les commandes déjà passées.

8.4 Règles relatives au panier et à la commande

- Il est impossible de commander un produit dont le statut est masqué ou dont le stock disponible est nul.
- Toute tentative d'ajout au panier ou de commande avec une quantité nulle ou négative est rejetée par validation Bean Validation.
- Lors de la validation d'une commande, le stock du ou des produits concernés est réservé temporairement, dans l'attente de la confirmation du paiement.
- La réservation de stock expire automatiquement après un délai défini si le paiement n'a pas été confirmé dans cet intervalle ; le stock réservé est alors restitué et redevient disponible à la vente.
- Le prix unitaire appliqué à chaque ligne de commande est déterminé et figé au moment de la validation de la commande ; il n'est jamais recalculé rétroactivement, y compris en cas de modification ultérieure du prix du produit.

8.5 Règles relatives au paiement

- Le stock d'un produit n'est décrémenté définitivement qu'après confirmation effective du paiement, et non lors de la simple validation du panier.
- En cas d'échec du paiement, la réservation de stock associée est immédiatement annulée et le stock correspondant est restitué au catalogue.
- Une commande ne peut avoir qu'un seul paiement validé à la fois ; toute tentative de double paiement pour une même commande déjà payée est rejetée.

8.6 Règles relatives à la facturation

- La facture n'est générée par le système qu'après confirmation d'un paiement validé ; aucune facture n'est émise pour une commande non payée.
- Le montant et les prix unitaires inscrits sur une facture sont définitifs et ne peuvent plus être modifiés une fois celle-ci générée.
- Chaque facture dispose d'un numéro unique, distinct du numéro de commande, permettant sa traçabilité comptable.

8.7 Autres règles générales

- Toute action d'écriture (création, modification, suppression) requiert un jeton JWT valide, à l'exception des opérations explicitement publiques de consultation du catalogue.
- Les opérations réservées à l'administrateur sont rejetées avec un code 403 si elles sont tentées par un compte de rôle Utilisateur.
- Toute donnée temporelle (dates de création, de modification, de commande, de paiement) est enregistrée au format horodaté UTC afin d'éviter toute ambiguïté liée aux fuseaux horaires.

9. Sécurité

La sécurité de la plateforme repose sur une combinaison de mécanismes appliqués à différents niveaux de l'architecture, depuis l'authentification jusqu'à la validation fine des données entrantes, en passant par la protection contre les attaques applicatives les plus courantes.

9.1 JWT et Refresh Token

L'authentification repose sur des jetons JWT signés, émis par le User/Auth Service lors de la connexion. Le jeton d'accès a une durée de vie volontairement courte afin de limiter les conséquences d'une éventuelle interception. Un jeton de rafraîchissement (refresh token), à durée de vie plus longue, permet d'obtenir un nouveau jeton d'accès sans nécessiter une nouvelle saisie des identifiants, tout en conservant la possibilité de révoquer une session en invalidant ce jeton.

9.2 BCrypt

Tous les mots de passe sont chiffrés à l'aide de l'algorithme BCrypt avant tout enregistrement en base de données. BCrypt intègre un sel aléatoire propre à chaque mot de passe, ce qui empêche l'utilisation de tables précalculées (rainbow tables) en cas de compromission de la base.

9.3 HTTPS

En environnement de production, l'ensemble des échanges entre le frontend, l'API Gateway et les microservices doit être réalisé exclusivement via HTTPS, garantissant la confidentialité et l'intégrité des données transmises, en particulier les identifiants de connexion et les jetons JWT.

9.4 Validation des entrées (Bean Validation)

Chaque donnée reçue par une API est validée à l'aide des annotations Bean Validation (telles que @NotNull, @Size, @Email, @Positive) directement sur les objets de transfert de données, avant tout traitement métier. Toute requête ne respectant pas ces contraintes est rejetée avec un code 400 accompagné d'un message d'erreur détaillant les champs concernés.

9.5 CORS

L'API Gateway est configurée avec une politique CORS restrictive, n'autorisant que les origines correspondant au frontend Angular officiel de la plateforme, afin d'empêcher qu'un site tiers non autorisé ne puisse interroger l'API depuis le navigateur d'un utilisateur authentifié.

9.6 Gestion des rôles et permissions

Trois rôles sont définis : VISITEUR (implicite, absence de jeton), UTILISATEUR et ADMIN. Chaque endpoint protégé déclare explicitement, via Spring Security, les rôles autorisés à y accéder. Cette vérification est réalisée à deux niveaux : une première fois au niveau de l'API Gateway pour un filtrage global, puis une seconde fois au niveau de chaque microservice, qui ne fait jamais confiance aveuglément à la couche amont.

9.7 Expiration du token

Chaque jeton JWT intègre une date d'expiration (claim exp) vérifiée systématiquement à chaque requête. Un jeton expiré est rejeté avec un code 401, invitant le client à utiliser son jeton de rafraîchissement ou à se reconnecter.

9.8 Journalisation de sécurité

Les tentatives de connexion, réussies comme échouées, ainsi que les tentatives d'accès à une ressource sans autorisation suffisante, sont journalisées avec un niveau de détail suffisant pour permettre une analyse a posteriori en cas d'incident de sécurité, sans toutefois journaliser de données sensibles telles que les mots de passe.

9.9 Protection contre l'injection SQL

L'utilisation systématique de Spring Data JPA et d'Hibernate, avec des requêtes paramétrées, élimine par construction le risque d'injection SQL, aucune requête n'étant construite par concaténation de chaînes de caractères issues des entrées utilisateur.

9.10 Protection contre les attaques XSS

Le frontend Angular échappe par défaut tout contenu dynamique inséré dans le DOM, ce qui limite fortement le risque d'exécution de scripts malveillants injectés via des champs texte (nom de produit, description, avis). Les données affichées provenant du backend sont systématiquement traitées comme du texte, jamais interprétées comme du HTML brut sans échappement explicite.

9.11 Protection CSRF

L'architecture reposant sur une authentification par jeton JWT transmis dans l'en-tête HTTP Authorization, et non par cookie de session, le risque de falsification de requête intersite (CSRF) est intrinsèquement réduit, ce mécanisme d'attaque exploitant principalement l'authentification par cookies automatiquement joints par le navigateur.

9.12 Validation des entrées utilisateur

Au-delà de la validation syntaxique assurée par Bean Validation, une validation métier complémentaire est réalisée au niveau de la couche service : par exemple, la vérification qu'une quantité commandée ne dépasse pas le stock disponible, ou qu'une adresse email suit un format cohérent avant tout envoi de communication.

9.13 Politique de mots de passe

Le système impose une longueur minimale et une combinaison de caractères pour tout mot de passe créé ou modifié, refusant les mots de passe jugés trop faibles dès la phase d'inscription ou de changement de mot de passe.

9.14 Verrouillage temporaire des comptes

Après un nombre défini de tentatives de connexion échouées consécutives sur un même compte, celui-ci est verrouillé temporairement, empêchant toute nouvelle tentative de connexion pendant une durée déterminée, ce qui limite l'efficacité d'attaques par force brute sur les identifiants.

10. Contraintes techniques

Cette section précise les contraintes techniques imposées par le choix des technologies retenues, ainsi que les conventions adoptées par le groupe afin de garantir la cohérence du code produit par les cinq membres de l'équipe.

10.1 Java et Spring

L'ensemble des microservices est développé en Java 21 avec Spring Boot 3, tirant parti des fonctionnalités récentes du langage. L'injection de dépendances et l'inversion de contrôle, principes fondamentaux du framework Spring, sont utilisées systématiquement : aucune instanciation manuelle de service ou de repository n'est réalisée dans le code métier, celle-ci étant entièrement déléguée au conteneur Spring.

10.2 Angular

Le frontend est développé avec Angular 20, en s'appuyant sur une architecture par modules et par composants autonomes (standalone components), avec une séparation claire entre les services d'accès aux données (communication HTTP avec l'API Gateway) et les composants de présentation.

10.3 Docker et Docker Compose

Chaque microservice dispose de son propre Dockerfile, construit à partir d'une image de base légère (JDK 21). Un unique fichier docker-compose.yml, placé à la racine du dépôt, orchestre le démarrage de l'API Gateway, des quatre microservices et de leurs quatre bases PostgreSQL respectives, avec des volumes dédiés pour la persistance des données.

10.4 GitHub

Le code source est hébergé sur un dépôt GitHub organisé en un dépôt par microservice ou en un mono-dépôt structuré par dossiers, selon le choix retenu par l'équipe, avec une branche principale protégée et des branches de fonctionnalité pour chaque évolution significative, fusionnées après relecture.

10.5 PostgreSQL

Chaque base de données PostgreSQL est versionnée indépendamment via les migrations gérées par Hibernate (ou un outil de migration dédié), garantissant que le schéma de chaque base évolue de façon traçable et reproductible sur tous les postes de développement.

10.6 OpenFeign et Resilience4J

Les interfaces de clients OpenFeign sont regroupées dans un package dédié au sein de chaque microservice consommateur, avec une configuration explicite des délais d'attente (timeouts) et des mécanismes Resilience4J associés à chaque appel, afin de garantir un comportement prévisible en cas de défaillance d'un service distant.

10.7 Swagger/OpenAPI et Postman

Chaque microservice expose sa documentation Swagger/OpenAPI sur un chemin dédié, générée automatiquement à partir des annotations des contrôleurs REST. Une collection Postman par microservice est maintenue à jour et versionnée dans le dépôt, permettant à chaque membre de l'équipe de tester rapidement les endpoints sans dépendre du frontend.

10.8 Convention de nommage

- Packages Java en minuscules, structurés par couche (controller, service, repository, entity, dto, config, exception).
- Classes en PascalCase, méthodes et variables en camelCase, constantes en MAJUSCULES_AVEC_UNDERSCORES.
- Tables et colonnes de base de données en snake_case, au singulier ou au pluriel de manière cohérente au sein d'un même service.
- Endpoints REST en kebab-case lorsqu'ils comportent plusieurs mots (par exemple /prix-gros).

10.9 Structure des projets et organisation des packages

Chaque microservice Spring Boot suit une structure de packages homogène : un package controller exposant les endpoints REST, un package service portant la logique métier, un package repository pour l'accès aux données via Spring Data JPA, un package entity pour les entités JPA, un package dto pour les objets de transfert exposés par l'API, un package config pour la configuration (sécurité, OpenFeign, Swagger), et un package exception pour la gestion centralisée des erreurs.

11. Diagrammes UML (description textuelle)

Conformément aux consignes du projet, les diagrammes UML ne sont pas représentés graphiquement dans ce document mais décrits textuellement, afin de permettre leur reconstruction ultérieure lors de la phase de modélisation détaillée.

11.1 Diagramme de cas d'utilisation

Acteur : Visiteur

Le visiteur est associé aux cas d'utilisation suivants : Consulter les catégories, Consulter les produits, Rechercher un produit, Filtrer les produits, Voir le détail d'un produit. Le cas d'utilisation « Ajouter au panier », bien que déclenché par le visiteur, inclut systématiquement le cas d'utilisation « S'authentifier », matérialisant la redirection obligatoire vers la connexion.

Acteur : Utilisateur

L'utilisateur hérite de tous les cas d'utilisation du visiteur et y ajoute : S'inscrire, Se connecter, Modifier son profil, Modifier son mot de passe, Ajouter un produit au panier, Modifier une quantité, Supprimer un article du panier, Passer une commande, Choisir une adresse de livraison, Choisir un mode de paiement, Effectuer un paiement, Consulter ses commandes, Consulter une facture, Télécharger une facture, Consulter son historique.

Acteur : Administrateur

L'administrateur dispose de cas d'utilisation spécifiques regroupés en plusieurs blocs : Gérer les produits (ajouter, modifier, supprimer, masquer, afficher, réapprovisionner), Gérer les prix de gros (ajouter, modifier, supprimer, activer, désactiver), Gérer les catégories (ajouter, modifier, supprimer, masquer, afficher), Gérer les commandes (consulter, modifier le statut), Gérer les utilisateurs (consulter, activer, désactiver), et Consulter le tableau de bord.

11.2 Diagramme de classes

Le diagramme de classes global, réparti conceptuellement selon les quatre microservices, s'articule autour des classes suivantes.

Domaine User/Auth

La classe Utilisateur porte les attributs id, nom, prenom, email, password, telephone, adresse, avatarUrl, profil, isActive, dateCreation et dateModification. Elle est associée à une énumération Role (VISITEUR, UTILISATEUR, ADMIN) via l'attribut profil.

Domaine Catalog

La classe Categorie (id, nom, description, statut) entretient une relation de composition « un-à-plusieurs » avec la classe Produit (id, nom, description, prix, stock, statut, imagePrincipale, imagesSecondaires, dateCreation, dateModification). La classe Produit entretient à son tour une relation « un-à-plusieurs » avec la classe PrixGros (id, quantiteMinimale, prix, statut), chaque produit pouvant posséder plusieurs paliers de prix de gros.

Domaine Order

La classe Panier (id, utilisateurId, dateCreation) est composée de plusieurs PanierItem (id, produitId, quantite, prixUnitaire). La classe Commande (id, numeroCommande, utilisateurId, dateCommande, etat, adresseLivraison, modePaiement, montantTotal) est composée de plusieurs LigneCommande (id, produitId, quantite, prixUnitaire) et est associée en « un-à-un » à une Facture (id, numeroFacture, dateEmission, montantTotal).

Domaine Payment

La classe Paiement (id, commandId, montant, modePaiement, statut, datePaiement) est associée logiquement, par référence d'identifiant, à la classe Commande du domaine Order, sans lien de clé étrangère physique entre les deux bases de données.

12. Parcours utilisateur

Cette section illustre, à travers des scénarios concrets, l'enchaînement des étapes fonctionnelles et techniques traversant les différents microservices pour aboutir à un résultat métier complet.

12.1 Scénario : achat d'un produit par un visiteur

Le parcours débute lorsqu'un visiteur, non authentifié, accède à la page d'accueil du site et consulte librement le catalogue des produits, exposé par le Catalog Service via l'API Gateway. Il affine sa recherche à l'aide des filtres par catégorie et par prix, puis consulte la fiche détaillée d'un produit qui l'intéresse.

Souhaitant acquérir ce produit, il clique sur le bouton « Ajouter au panier ». Le frontend Angular détecte l'absence de jeton JWT valide et redirige immédiatement le visiteur vers l'écran de connexion, en conservant en mémoire le produit et la quantité initialement sélectionnés. Le visiteur choisit de créer un compte, renseigne les informations requises, puis le User/Auth Service valide et enregistre son inscription avant de lui délivrer un jeton JWT.

Une fois authentifié, l'action initialement interrompue est rejouée automatiquement : le produit est ajouté à son panier via l'Order Service. L'utilisateur poursuit ensuite ses achats ou se rend directement sur son panier, où il peut ajuster les quantités. Il valide ensuite sa commande : l'Order Service vérifie, via un appel OpenFeign au Catalog Service, la disponibilité du stock et le prix applicable (normal ou de gros selon la quantité), puis procède à la réservation temporaire du stock correspondant.

L'utilisateur choisit son adresse de livraison ainsi que son mode de paiement, puis initie le paiement. L'Order Service transmet la demande au Payment Service, qui traite la transaction et retourne son résultat. En cas de succès, le Payment Service notifie l'Order Service, qui confirme la commande, décrémente définitivement le stock via le Catalog Service, et génère automatiquement la facture correspondante. L'utilisateur peut alors consulter et télécharger sa facture, ainsi que retrouver cette commande dans son historique.

En cas d'échec du paiement, l'Order Service est notifié de ce rejet, annule la réservation de stock associée et restitue les quantités réservées au Catalog Service, tout en informant l'utilisateur de l'échec et en l'invitant à réessayer avec un autre mode de paiement.

12.2 Scénario : gestion administrative d'un produit

L'administrateur se connecte à la plateforme avec un compte disposant du rôle ADMIN. Il accède à l'espace d'administration, où le tableau de bord lui présente une synthèse des indicateurs clés de la plateforme (nombre de produits, de commandes, chiffre d'affaires, utilisateurs actifs).

Il se rend ensuite dans la gestion des produits, où il constate qu'un article est en rupture de stock. Il sélectionne le produit concerné et procède à son réapprovisionnement en indiquant la quantité reçue ; le Catalog Service met immédiatement à jour le stock disponible, rendant le produit à nouveau commandable pour les utilisateurs.

L'administrateur souhaite ensuite encourager les achats en volume sur ce produit : il ajoute deux paliers de prix de gros, l'un à partir de dix unités, l'autre à partir de vingt unités, chacun associé à un tarif dégressif. Ces paliers sont immédiatement pris en compte par le Catalog Service lors du calcul du prix pour toute nouvelle commande.

Enfin, l'administrateur consulte la liste des commandes en attente d'expédition, sélectionne celles qui ont été physiquement préparées, et fait progresser leur statut vers « Expédiée » via l'interface de gestion des commandes, ce qui met à jour l'état correspondant dans l'Order Service et informe les utilisateurs concernés.

13. Répartition du travail

Le projet est réalisé par un groupe de cinq étudiants. La répartition des tâches a été établie de manière à ce que chaque microservice métier dispose d'un responsable clairement identifié, garant de sa conception, de son développement et de sa documentation, tandis que la cinquième personne du groupe assure la responsabilité transversale du frontend Angular et de la coordination des intégrations.

13.1 Répartition des responsabilités

Adama Seck — User/Auth Service

Adama Seck est responsable de la conception et du développement complet du microservice de gestion des utilisateurs et de l'authentification : modélisation de l'entité Utilisateur, mise en place de l'inscription et de la connexion, génération et validation des jetons JWT, gestion des rôles, gestion du profil et du mot de passe, ainsi que la sécurisation générale de ce service avec Spring Security et BCrypt.

Aissatou Diagne — Catalog Service

Aissatou Diagne est responsable du microservice de gestion du catalogue : modélisation des entités Produit, Catégorie et PrixGros, développement des fonctionnalités de recherche et de filtrage, mise en œuvre du mécanisme de calcul automatique du tarif applicable selon les paliers de prix de gros, et gestion du cycle de vie du stock (réservation, restitution, décrétement).

Arame Bacar Cissé — Order Service

Arame Bacar Cissé est responsable du microservice de gestion des commandes : modélisation du panier, de la commande, des lignes de commande et de la facture, mise en œuvre du processus de validation de commande incluant l'appel au Catalog Service pour la vérification du stock et du tarif, ainsi que la génération automatique des factures après confirmation du paiement.

Racine Sabaly — Payment Service

Racine Sabaly est responsable du microservice de gestion des paiements : modélisation de l'entité Paiement, mise en œuvre du processus de validation ou de rejet d'une transaction, et notification du résultat à l'Order Service via un appel OpenFeign, en garantissant la cohérence entre le statut du paiement et celui de la commande associée.

Abdoulaye Niang — Frontend Angular, API Gateway et coordination

Abdoulaye Niang est responsable du développement de l'application Angular dans son ensemble : intégration avec l'API Gateway, réalisation des interfaces de consultation du catalogue, du panier, du tunnel de commande et des tableaux de bord administrateur, ainsi que la mise en place de l'intercepteur HTTP gérant l'attache automatique du jeton JWT. Il assure en outre la coordination des intégrations entre les quatre microservices développés par ses camarades et la conduite des tests d'intégration frontend-backend.

13.2 Dépendances entre les membres

La nature distribuée du projet impose un ordre logique de disponibilité de certaines briques avant que d'autres puissent être pleinement testées de bout en bout, même si chaque membre peut développer son microservice de façon largement autonome grâce à l'indépendance des bases de données.

- Le Catalog Service (Aissatou Diagne) doit exposer un socle fonctionnel minimal (liste des produits, disponibilité) avant que l'Order Service (Arame Bacar Cissé) ne puisse implémenter et tester la vérification de stock lors de la validation d'une commande.
- Le User/Auth Service (Adama Seck) doit être opérationnel, en particulier l'émission de jetons JWT, avant que l'API Gateway ne puisse valider les jetons transmis par le frontend.

- L'Order Service doit exposer un socle fonctionnel minimal avant que le Payment Service (Racine Sabaly) ne puisse tester la notification du résultat d'un paiement vers l'Order Service.
- Abdoulaye Niang dépend de la disponibilité progressive des contrats d'API (endpoints, formats de requêtes et réponses) de chacun des quatre microservices pour intégrer les interfaces correspondantes ; une documentation Swagger à jour de chaque service est donc un prérequis à son travail d'intégration.

13.3 Livrables attendus par membre

Membre	Livrables attendus
Adama Seck	Microservice User/Auth conteneurisé, documentation Swagger, collection Postman, tests unitaires
Aissatou Diagne	Microservice Catalog conteneurisé, documentation Swagger, collection Postman, tests unitaires
Arame Bacar Cissé	Microservice Order conteneurisé, documentation Swagger, collection Postman, tests unitaires
Racine Sabaly	Microservice Payment conteneurisé, documentation Swagger, collection Postman, tests unitaires
Abdoulaye Niang	Application Angular complète, configuration API Gateway, rapport de tests d'intégration

13.4 Planning logique de développement

Le développement est organisé en quatre phases successives. La première phase, consacrée à la mise en place, voit chaque responsable initialiser son microservice (structure de projet, connexion à sa base PostgreSQL, premières entités) tandis qu'Abdoulaye Niang met en place le squelette du projet Angular et la configuration initiale de l'API Gateway.

La deuxième phase est consacrée au développement des fonctionnalités cœur de chaque service de façon indépendante : authentification complète côté User/Auth, gestion des produits et catégories côté Catalog, gestion du panier côté Order, et structure de base du traitement des paiements côté Payment, en parallèle de la construction des premières interfaces Angular de consultation du catalogue.

La troisième phase porte sur l'intégration inter-services : mise en place des clients OpenFeign entre l'Order Service et le Catalog Service, puis entre l'Order Service et le Payment Service, ainsi que la connexion complète du frontend à l'ensemble des microservices via l'API Gateway, sous la coordination d'Abdoulaye Niang.

La quatrième et dernière phase est dédiée aux tests de bout en bout, à la correction des anomalies détectées lors des parcours utilisateurs complets, à la finalisation de la documentation Swagger et des collections Postman, ainsi qu'à la préparation de la démonstration finale du projet.

14. Conclusion

Le présent cahier des charges a permis de poser de façon exhaustive les fondations de la plateforme e-commerce que le groupe s'apprête à développer dans le cadre des modules de développement Backend avec Spring Boot et de développement Frontend avec Angular. Au-delà de la simple description d'une application marchande, ce document témoigne d'une réflexion approfondie sur les enjeux propres aux architectures distribuées : indépendance des microservices, isolation des bases de données selon le principe Database per Service, communication inter-services maîtrisée via OpenFeign, et résilience face aux pannes assurée par Resilience4J.

La distinction claire entre les trois profils d'utilisateurs — visiteur, utilisateur authentifié et administrateur — structure l'ensemble des besoins fonctionnels et garantit une expérience d'achat fluide, où la consultation du catalogue reste libre tandis que l'engagement transactionnel impose naturellement la création d'un compte. Cette logique, couplée à la gestion fine des stocks, des prix de gros et du cycle de vie complet d'une commande jusqu'à la facturation, constitue le cœur de la valeur métier de la plateforme.

Sur le plan organisationnel, la répartition des responsabilités entre les cinq membres du groupe, chacun garant d'un microservice ou du frontend, ainsi que l'identification explicite des dépendances entre les différentes briques, doit permettre une progression coordonnée du projet tout en respectant l'autonomie de chaque contributeur, conformément à l'esprit même d'une architecture microservices.

Ce document constitue ainsi la référence commune à laquelle chaque membre de l'équipe pourra se reporter tout au long du développement, aussi bien pour la modélisation des données que pour la définition des contrats d'API, des règles métier ou des exigences de sécurité. Il pourra, au besoin, être complété ou affiné au fil de l'avancement du projet, notamment lors de la phase de conception détaillée et de la rédaction des diagrammes UML définitifs.